# NMC-Opt Studio: Compiler Analytical Cost Model & Target Placement Methodology

Rigorous Formulation of Data-Movement-Aware Target Selection for Near-Memory Computing

**Author / Research Team:** Compiler Design & Computer Systems Engineering Group
**Artifact Status:** Formal Academic Specification & Research Methodology Document
**Repository Source:** https://github.com/MSN-1307/NMC-OPT-CD | **Target Architecture:** Host CPU vs 16-Bank PIM

## 1. Executive Problem Framing: The Memory Wall in Modern Compilers

Modern high-performance compilers (e.g., GCC, LLVM) employ sophisticated intermediate representations and loop optimization passes designed primarily around arithmetic operation counts, instruction-level parallelism (ILP), and register pressure. However, in memory-bound scientific and machine learning workloads, performance and energy consumption are overwhelmingly dominated by off-chip data movement across physical interconnects (PCIe Gen5, CXL 3.0, and DDR5 multi-channel buses). This discrepancy—the classical **Memory Wall**—imposes a critical throughput barrier for kernels with low arithmetic intensity.

Near-Memory Computing (NMC) and Processing-In-Memory (PIM) architectures place Data Processing Units (DPUs) directly adjacent to DRAM banks, offering ultra-wide aggregate memory bandwidth (e.g., 160+ GB/s across 16 internal banks) at very low physical energy costs. Despite these hardware capabilities, modern optimizing compilers lack native analytical cost models to autonomously partition loops between conventional host CPUs and in-memory DPUs. NMC-Opt Studio formalizes data movement as a first-class compiler optimization metric.

## 2. End-to-End Compiler Optimization Pipeline

The compiler executes a deterministic seven-stage analytical pipeline that transforms raw C source loops into target placement decisions:

| Stage | Pipeline Phase | Compiler Functional Responsibility |
|---|---|---|
| 1 | **Source Intake** | Accepts canonical C loop constructs, function prototypes, and pragmas. |
| 2 | **Syntactic Validation** | Runs pycparser AST syntax validation; verifies single-loop bound constraints. |
| 3 | **AST Loop Profiling** | Computes trip counts, memory footprint (bytes read/written), and Arithmetic Intensity. |
| 4 | **SSA IR Generation** | Lowers loop bodies into 3-address SSA quadruples (gep, load, store, fadd, fmul). |
| 5 | **Cost Model Evaluation** | Applies first-principles equations to calculate T_cpu and T_nmc in milliseconds. |
| 6 | **Placement Matrix** | Evaluates baseline and transformed candidate strategies (Fusion, Tiling). |
| 7 | **Optimization Verdict** | Emits definitive offload target, expected speedup factor, and bottleneck audit. |

## 3. Analytical Cost Model Formulation

All component execution latencies are formulated in **milliseconds (ms)** using explicit dimensional conversions (1 GFLOPS = 10^6 ops/ms; 1 GB/s = 10^6 B/ms):

```
// A. Host CPU Execution Latency (ms)
T_cpu = T_comp_cpu + T_mem_cpu
T_comp_cpu = W / (Pi_cpu * 1e6)
T_mem_cpu  = D / (Beta_cpu * 1e6)

// B. Near-Memory Computing (NMC) Offload Latency (ms)
T_nmc = T_comp_nmc + T_mem_nmc + T_link + T_layout + T_sync
T_comp_nmc = W / (Pi_nmc * 1e6)
T_mem_nmc  = D / (Beta_nmc * 1e6)
T_link     = (D_read + D_write) / (Beta_link * 1e6) + t_h2d + t_d2h
T_layout   = (D_read * alpha) / (Beta_layout * 1e6) + t_layout
T_sync     = S * t_barrier
```

## 4. Transformation Legality & Placement Decision Rule

The compiler defines explicit legality preconditions before evaluating candidate transformations:

- **Loop Fusion (NMC + Loop Fusion):** Legal only when consecutive loops share identical trip counts and produce point-to-point element-wise outputs without loop-carried backward dependencies. Eliminates intermediate DRAM round trips.
- **Loop Tiling (NMC + Tiling):** Legal exclusively on multi-dimensional loop nests (e.g., GEMV). Partitions matrix rows across independent DRAM banks, keeping vector elements resident in bank-local scratchpad memories.

***The Core Compiler Placement Decision Rule:***
*Offload to In-Memory DPUs ONLY IF **T_nmc < T_cpu** (which occurs when Arithmetic Intensity **I < I_crossover**).*
*If I ≥ I_crossover, compute overhead dominates over memory transfer savings, and the compiler retains the loop on the Host CPU.*

## 5. 16-Bank Interleaved Memory Layout & Contention Model

Physical DRAM is organized into multiple independent banks. Under naive contiguous allocation, stride-1 vector reads across parallel DPUs cause acute channel collisions and row-buffer conflicts. NMC-Opt Studio models bank interleaving where contiguous memory addresses are striped across 16 independent banks: *Bank_ID = (Address / Cache_Line_Size) mod 16*. This achieves peak aggregate internal throughput (160 GB/s) while charging an explicit compile-time layout transformation penalty (T_layout) for unaligned input data.

## 6. Research Foundations & Empirical Grounding

NMC-Opt Studio's analytical models are validated against published findings in high-performance memory-centric literature:

1. **IMTP (arXiv:2412.19630, 2024):** Validates compiler-level host/PIM boundary management, bank-local scratchpad buffering, and boundary check overheads on UPMEM-like DRAM architectures.
2. **Singh et al. (Microprocessors & Microsystems, 2019):** Confirms the 5x–10x internal memory bandwidth advantage of 3D-stacked and 2D DRAM PIM over host memory channels.
3. **Classical Roofline Model (Williams et al., 2009):** Provides the theoretical operational intensity bound (FLOP/Byte) that governs the transition between memory-bound and compute-bound regimes.